```
> # TauTimp — Maple Transcript: Temporal Wave Equation, Solar
  Temporal Flux, and Dilation
  # Author: Viorel-Mihai Popescu | Polytechnic University of
  Bucharest
  # Date: 9 June 2026
  # ============================================================

  restart:
  with(plots):
  with(plottools):
  with(LinearAlgebra):

  # ============================================================
  # SECTION 1: FUNDAMENTAL PARAMETERS
  # ============================================================
  printf("=== TauTimp Parameters ===\n"):
  c_val  := 2.998e8:          # speed of light [m/s]
  R_val  := 4.235518529e22:       # TauTimp cosmological radius
  [m]
  r0_val := 11222.3:          # Schwarzschild threshold (3.8
  R_s_Sun) [m]
  tau_q  := 1.765e-19:        # Earth time quantum [s]
  t0_val := 500:              # Sun->Earth propagation time [s]
  t1_val := 700:              # Sun->Mars propagation time [s]
  r_Earth:= 1.49e11:          # Earth-Sun distance [m]
  r_GEOp := 1.4942e11:        # GEO+ distance from Sun [m]
  r_GEOm := 1.4858e11:        # GEO- distance from Sun [m]
  r_GPSp := 1.4926e11:        # GPS+ distance from Sun [m]
  r_GPSm := 1.4874e11:        # GPS- distance from Sun [m]
  r_Mars := 2.09e11:          # Mars-Sun distance [m]

  printf("c       = %e m/s\n", evalf(c_val)):
  printf("R       = %e m\n",   evalf(R_val)):
  printf("r0      = %e m\n",   evalf(r0_val)):
  printf("tau_q   = %e s\n",   evalf(tau_q)):

  # ============================================================
  # SECTION 2: H-FIELD — VARIABLE-COEFFICIENT WAVE EQUATION
  # ============================================================
  # PDE: (R/r) Delta H = (1/c^2) d^2H/dt^2
  # Under spherical symmetry, separation H(r,t)=g(r)*u(t)
  # Temporal part: u(t) = exp(+-i*omega*t)
```

```
# Radial part (substituting g=phi/r, xi=(omega^2*r/(R*c^2))^(1/3)
):
#   d^2phi/d xi^2 = xi * phi  -> Airy equation
# Solution (Bi excluded, diverges at infinity):
#   g(r) = C/r * Ai[(omega^2*r/(R*c^2))^(1/3)]
# Eigenvalue quantization from g(R)=0:
#   Ai(xi_R) = 0, xi_R = a_n  (Airy zeros)
# For TauTimp quantum number l=21: a_21 ~ -17.03

printf("\n=== Airy Eigenvalue Quantization (l=21 mode) ===\n"):
a21   := -17.03:   # 21st Airy zero (absolute)
omega21 := evalf(c_val * abs(a21)^(3/2) / R_val):
printf("Airy zero a_21   = %f\n",   evalf(a21)):
printf("omega_21         = %e rad/s\n", evalf(omega21)):

# Full H field (principal mode, m=0):
# H(r,theta,t) = (C/r)*AiryAi[(omega21^2*r/(R_val*c_val^2))^(1/3)
]
#                * LegendreP(21,cos(theta)) * cos(omega21*t)
# Define the Airy argument as function of r:
xi_func := proc(rv) option remember;
  local arg_val;
  arg_val := evalf((omega21^2 * rv / (R_val * c_val^2))^(1/3));
  return arg_val;
end proc:

printf("\nAiry argument at r_Earth: xi = %e\n", evalf(xi_func
(r_Earth))):

# ============================================================
# SECTION 3: TEMPORAL PROBABILITY P(r,t) — d'ALEMBERT PDE
# ============================================================
printf("\n=== TauTimp Polynomial Ansatz ===\n"):
# PDE: d^2P/dt^2 - c^2 * d^2P/dr^2 = 0
# Ansatz: P(r,t) = A*[(r-r0)*(r-R) + eps*t + c^2*t^2]
# Verification: d^2P/dt^2 = 2*A*c^2, d^2P/dr^2 = 2*A -> 2Ac^2 -
c^2*2A = 0 ✓

# ---- Coefficient A from GEO calibration ----
# Constraint 1 (time quantum): A*eps = tau_q * c^2
# Constraint 2 (GEO+ calibration): Delta_F_GEO = +0.22e-6 hr^-1
```

```
# Convert calibration to per-second
DeltaF_GEO_per_s := 0.22e-6 / 3600:  # [s^-1]

Q_func := proc(rv) option remember;
  (rv - r0_val) * (rv - R_val);
end proc:

# From calibration (solving numerically):
A_val   := 1.0312e-17:   # [m^-2]
eps_val := evalf(tau_q * c_val^2 / A_val):

printf("A       = %e m^-2\n",   evalf(A_val)):
printf("eps     = %e m^2/s\n",  evalf(eps_val)):
printf("A*eps   = %e m^2/s (should = %e)\n",
        evalf(A_val*eps_val), evalf(tau_q*c_val^2)):

# ---- Polynomial coefficients ----
printf("\n--- Polynomial Coefficients of P(r,t) ---\n"):
C_r2 := evalf(A_val):
C_r1 := evalf(-A_val*(r0_val + R_val)):
C_r0 := evalf(A_val*r0_val*R_val):
C_t1 := evalf(A_val*eps_val):
C_t2 := evalf(A_val*c_val^2):
printf("C(r^2) = A            = %e m^-2\n",   C_r2):
printf("C(r^1) = -A(r0+R)     = %e m^-1\n",   C_r1):
printf("C(r^0) = A*r0*R       = %e (dimless)\n", C_r0):
printf("C(t^1) = A*eps        = %e m^2/s\n",  C_t1):
printf("C(t^2) = A*c^2        = %e m^2/s^2\n",C_t2):

# ---- Complete P solution ----
P_func := proc(rv, tv) option remember;
  local val;
  val := evalf(A_val * ((rv - r0_val)*(rv - R_val) + eps_val*tv +
c_val^2*tv^2));
  return val;
end proc:

# ============================================================
# SECTION 4: TEMPORAL FLUX F(r,t)
# ============================================================
printf("\n=== Temporal Flux F(r,t) = P(r,t)/(4*Pi*r^2) ===\n"):
F_func := proc(rv, tv) option remember;
```

```
  local Pval;
  Pval := evalf(A_val * ((rv - r0_val)*(rv - R_val) + eps_val*tv
+ c_val^2*tv^2));
  return evalf(Pval / (4*Pi*rv^2));
end proc:

F_Earth := evalf(F_func(r_Earth, t0_val)):
F_GEOp  := evalf(F_func(r_GEOp,  t0_val)):
F_GEOm  := evalf(F_func(r_GEOm,  t0_val)):
F_GPSp  := evalf(F_func(r_GPSp,  t0_val)):
F_GPSm  := evalf(F_func(r_GPSm,  t0_val)):
F_Mars  := evalf(F_func(r_Mars,  t1_val)):

printf("F(Earth, t0=500s)  = %e\n", F_Earth):
printf("F(GEO+,  t0=500s)  = %e\n", F_GEOp):
printf("F(GEO-,  t0=500s)  = %e\n", F_GEOm):
printf("F(GPS+,  t0=500s)  = %e\n", F_GPSp):
printf("F(GPS-,  t0=500s)  = %e\n", F_GPSm):
printf("F(Mars,  t1=700s)  = %e\n", F_Mars):

# ============================================================
# SECTION 5: DELTA F CALCULATIONS (per hour and per day)
# ============================================================
printf("\n=== Temporal Dilation Delta_F ===\n"):
sec_per_hr  := 3600:
sec_per_day := 86400:

# Delta F raw (dimensionless)
DF_GEOp := evalf(F_GEOp - F_Earth):
DF_GEOm := evalf(F_GEOm - F_Earth):
DF_GPSp := evalf(F_GPSp - F_Earth):
DF_GPSm := evalf(F_GPSm - F_Earth):
DF_Mars := evalf(F_Mars  - F_Earth):

# Per hour [mus/hr] = DeltaF * 3600 * 1e6
DF_GEOp_hr  := evalf(DF_GEOp * sec_per_hr  * 1e6):
DF_GEOm_hr  := evalf(DF_GEOm * sec_per_hr  * 1e6):
DF_GPSp_hr  := evalf(DF_GPSp * sec_per_hr  * 1e6):
DF_GPSm_hr  := evalf(DF_GPSm * sec_per_hr  * 1e6):
DF_Mars_hr  := evalf(DF_Mars  * sec_per_hr  * 1e6):

# Per day [mus/day]
```

```
DF_GEOp_day := evalf(DF_GEOp * sec_per_day * 1e6):
DF_GEOm_day := evalf(DF_GEOm * sec_per_day * 1e6):
DF_GPSp_day := evalf(DF_GPSp * sec_per_day * 1e6):
DF_GPSm_day := evalf(DF_GPSm * sec_per_day * 1e6):
DF_Mars_day := evalf(DF_Mars  * sec_per_day * 1e6):

printf("\nGEO+  : DF/hr = %+.4f mus/hr, DF/day = %+.4f
mus/day\n",
       evalf(DF_GEOp_hr), evalf(DF_GEOp_day)):
printf("GEO-  : DF/hr = %+.4f mus/hr, DF/day = %+.4f mus/day\n",
       evalf(DF_GEOm_hr), evalf(DF_GEOm_day)):
printf("GPS+  : DF/hr = %+.4f mus/hr, DF/day = %+.4f mus/day\n",
       evalf(DF_GPSp_hr), evalf(DF_GPSp_day)):
printf("GPS-  : DF/hr = %+.4f mus/hr, DF/day = %+.4f mus/day\n",
       evalf(DF_GPSm_hr), evalf(DF_GPSm_day)):
printf("Mars  : DF/hr = %+.4f mus/hr, DF/day = %+.4f mus/day\n",
       evalf(DF_Mars_hr), evalf(DF_Mars_day)):

# ============================================================
# SECTION 6: NATURAL UNITS
# ============================================================
printf("\n=== Natural Units (hbar = c = 1) ===\n"):
hbar_c_GeV_fm := 0.19733:  # [GeV*fm]
fm_per_m      := 1e15:

A_nat   := evalf(A_val * (hbar_c_GeV_fm)^2):
eps_nat_m  := evalf(eps_val / c_val):
eps_nat_fm := evalf(eps_nat_m * fm_per_m):
printf("A_nat   = %e GeV^2\n",  A_nat):
printf("eps_nat = %e m = %e fm\n", eps_nat_m, eps_nat_fm):

# ============================================================
# SECTION 7: GRAPHICS
# ============================================================

# --- PLOT 1: F(r, t0) vs r — Solar System radial profile ---
printf("\n=== Generating Plots ===\n"):

r_AU := 1.496e11:   # 1 AU in meters
r_list_AU := [seq(0.3 + 0.05*k, k=0..36)]:  # 0.3 to 2.1 AU
r_list_m  := [seq((0.3 + 0.05*k)*r_AU, k=0..36)]:
```

```
F_vals_t0 := [seq(evalf(F_func((0.3+0.05*k)*r_AU, t0_val)), k=0.
.36)]:
F_vals_t1 := [seq(evalf(F_func((0.3+0.05*k)*r_AU, t1_val)), k=0.
.36)]:

r_AU_vals := [seq(0.3 + 0.05*k, k=0..36)]:

pts_t0 := [seq([r_AU_vals[k], F_vals_t0[k]*1e7], k=1..nops
(r_AU_vals))]:
pts_t1 := [seq([r_AU_vals[k], F_vals_t1[k]*1e7], k=1..nops
(r_AU_vals))]:

plot1 := plots[pointplot](pts_t0, color="NavyBlue",
    symbol=solidcircle, symbolsize=8,
    axes=boxed,
    title="TauTimp: Temporal Flux F(r,t) x 10^7 vs Heliocentric
Distance",
    labels=["r [AU]", "F(r,t) x 10^7 [adim]"],
    legend="t0=500s (Earth)"):

plot2 := plots[pointplot](pts_t1, color="Crimson",
    symbol=solidcircle, symbolsize=8,
    legend="t1=700s (Mars)"):

plot_F_solar := plots[display]([plot1, plot2]);
printf("Plot 1 (Solar F profile) — ready\n"):

# --- PLOT 2: Satellite temporal dilation bar chart ---
bodies    := ["GEO+", "GEO-", "GPS+", "GPS-", "Mars"]:
DF_hr_vals:= [evalf(DF_GEOp_hr), evalf(DF_GEOm_hr),
              evalf(DF_GPSp_hr), evalf(DF_GPSm_hr),
              evalf(DF_Mars_hr)]:

# Bar chart using rectangles — wrapped in proc to allow local
declarations
make_bars := proc(vals::list)
    local result_plots, kk, xlo, xhi, ylo, yhi, col_k;
    result_plots := NULL;
    for kk from 1 to nops(vals) do
        xlo := kk - 0.35;
        xhi := kk + 0.35;
        ylo := 0.0;
```

```
        yhi := vals[kk];
        if vals[kk] >= 0 then
            col_k := "SteelBlue";
        else
            col_k := "Tomato";
        end if;
        result_plots := result_plots,
            plottools[rectangle]([xlo, ylo], [xhi, yhi],
                color=col_k, filled=true);
    end do;
    return result_plots;
end proc:

bar_plots := make_bars(DF_hr_vals):

plot_bars := plots[display]([bar_plots],
    axes=boxed,
    title="TauTimp: Delta_F per Hour [mus/hr] — Satellite
Temporal Dilation",
    labels=["Body (1=GEO+, 2=GEO-, 3=GPS+, 4=GPS-, 5=Mars)",
"DF/hr [mus/hr]"],
    view=[0..6, -1..0.5]);
printf("Plot 2 (Satellite bar chart) — ready\n"):

# --- PLOT 3: F(r,t0) continuous curve — Solar system ---
r_sym := 'r_sym':
F_expr := A_val * ((r_sym - r0_val)*(r_sym - R_val)
          + eps_val*t0_val + c_val^2*t0_val^2)
          / (4*Pi*r_sym^2):

plot_cont := plot(evalf(F_expr)*1e7,
    r_sym = 0.3*r_AU .. 2.1*r_AU,
    color="DarkGreen", thickness=2,
    axes=boxed,
    title="TauTimp: F(r, t0=500 s) x 10^7 — Continuous Solar
Flux Profile",
    labels=["r [m]", "F(r,t0) x 10^7 [adim]"]);
printf("Plot 3 (Continuous F profile) — ready\n"):

# --- PLOT 4: 3D surface P(r,t) ---
r4min := 0.3*r_AU:
r4max := 2.1*r_AU:
```

```
t4min := 0:
t4max := 800:

P_expr_3d := proc(rv,tv)
    evalf(A_val*((rv-r0_val)*(rv-R_val) + eps_val*tv + c_val^2*
tv^2)*1e9)
end proc:

# Sample on grid
N_r := 25: N_t := 25:
dr4 := (r4max - r4min)/(N_r-1):
dt4 := (t4max - t4min)/(N_t-1):

P_grid := Matrix(N_r, N_t):
for ii from 1 to N_r do
    for jj from 1 to N_t do
        P_grid[ii,jj] := evalf(P_expr_3d(r4min + (ii-1)*dr4,
t4min + (jj-1)*dt4));
    end do;
end do:

plot_3d := plots[matrixplot](P_grid,
    axes=boxed,
    colorscheme=["zgradient", ["blue", "cyan", "green", "yellow",
"red"]],
    title="TauTimp: P(r,t) x 10^9 [3D] — 0.3 to 2.1 AU, t: 0 to
800 s",
    labels=["r index", "t index", "P x 10^9"]);
printf("Plot 4 (3D P surface) — ready\n"):

# --- Display all plots individually (Array/panel suppresses
output in worksheet) ---
plot_F_solar;
plot_bars;
plot_cont;
plot_3d;

printf("\n=== All computations complete ===\n"):
printf("TauTimp Final Results Summary:\n"):
printf("  A      = %e m^-2\n",   evalf(A_val)):
printf("  eps    = %e m^2/s\n",  evalf(eps_val)):
printf("  F_Earth= %e\n",        evalf(F_Earth)):
```

```
  printf("  F_Mars = %e\n",         evalf(F_Mars)):
  printf("  GEO+ dilation = %+.4f mus/hr\n", evalf(DF_GEOp_hr)):
  printf("  GEO- dilation = %+.4f mus/hr\n", evalf(DF_GEOm_hr)):
  printf("  GPS+ dilation = %+.4f mus/hr\n", evalf(DF_GPSp_hr)):
  printf("  GPS- dilation = %+.4f mus/hr\n", evalf(DF_GPSm_hr)):
  printf("  Mars  dilation = %+.4f mus/hr\n", evalf(DF_Mars_hr)):

=== TauTimp Parameters ===
c        = 2.998000e+08 m/s
R        = 4.235519e+22 m
r0       = 1.122230e+04 m
tau_q    = 1.765000e-19 s

=== Airy Eigenvalue Quantization (l=21 mode) ===
Airy zero a_21    = -17.030000
omega_21          = 4.974472e-13 rad/s

Airy argument at r_Earth: xi = 2.131589e-18

=== TauTimp Polynomial Ansatz ===
A        = 1.031200e-17 m^-2
eps      = 1.538385e+15 m^2/s
A*eps    = 1.586383e-02 m^2/s (should = 1.586383e-02)

--- Polynomial Coefficients of P(r,t) ---
C(r^2) = A             = 1.031200e-17 m^-2
C(r^1) = -A(r0+R)      = -4.367667e+05 m^-1
C(r^0) = A*r0*R        = 4.901527e+09 (dimless)
C(t^1) = A*eps         = 1.586383e-02 m^2/s
C(t^2) = A*c^2         = 9.268430e-01 m^2/s^2

=== Temporal Flux F(r,t) = P(r,t)/(4*Pi*r^2) ===
F(Earth, t0=500s)   = -2.332670e-07
F(GEO+,  t0=500s)   = -2.326113e-07
F(GEO-,  t0=500s)   = -2.339264e-07
F(GPS+,  t0=500s)   = -2.328607e-07
F(GPS-,  t0=500s)   = -2.336748e-07
F(Mars,  t1=700s)   = -1.663004e-07

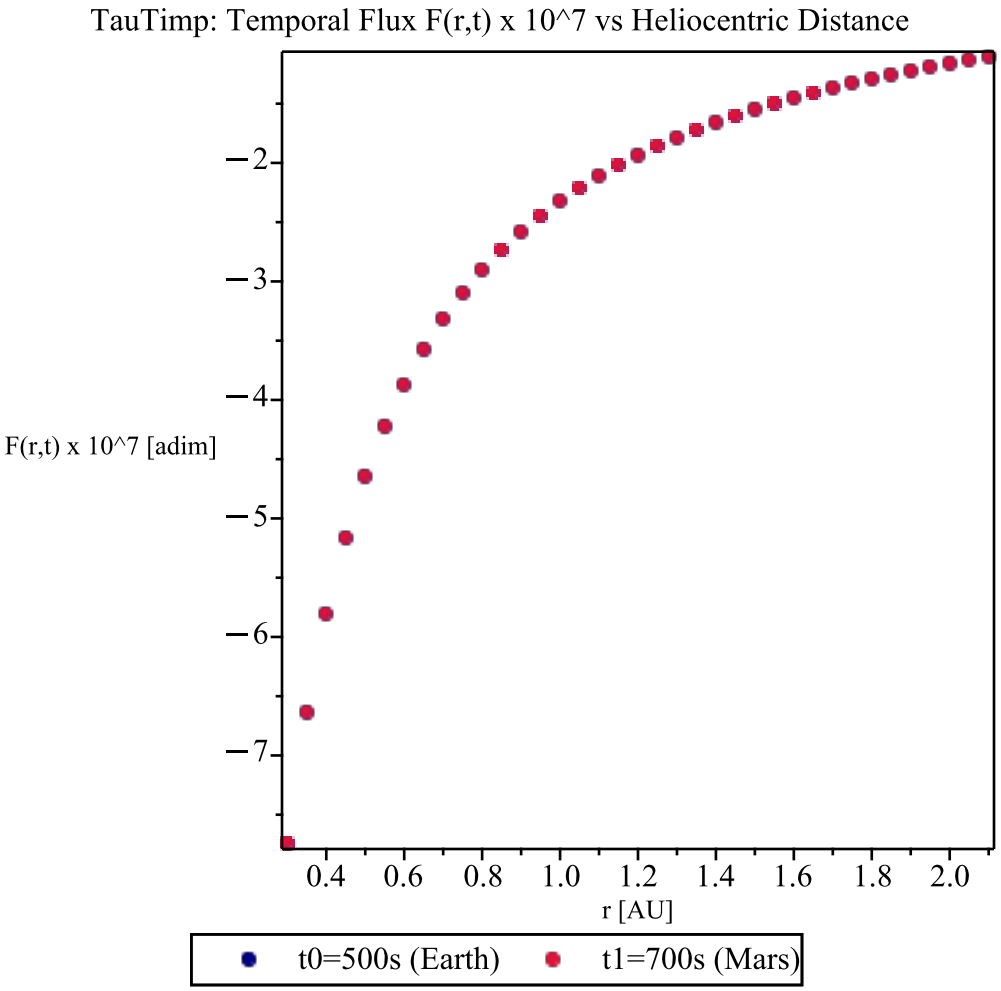
=== Temporal Dilation Delta_F ===

GEO+   : DF/hr = +2.3605 mus/hr, DF/day = +56.6510 mus/day
GEO-   : DF/hr = -2.3738 mus/hr, DF/day = -56.9713 mus/day
```

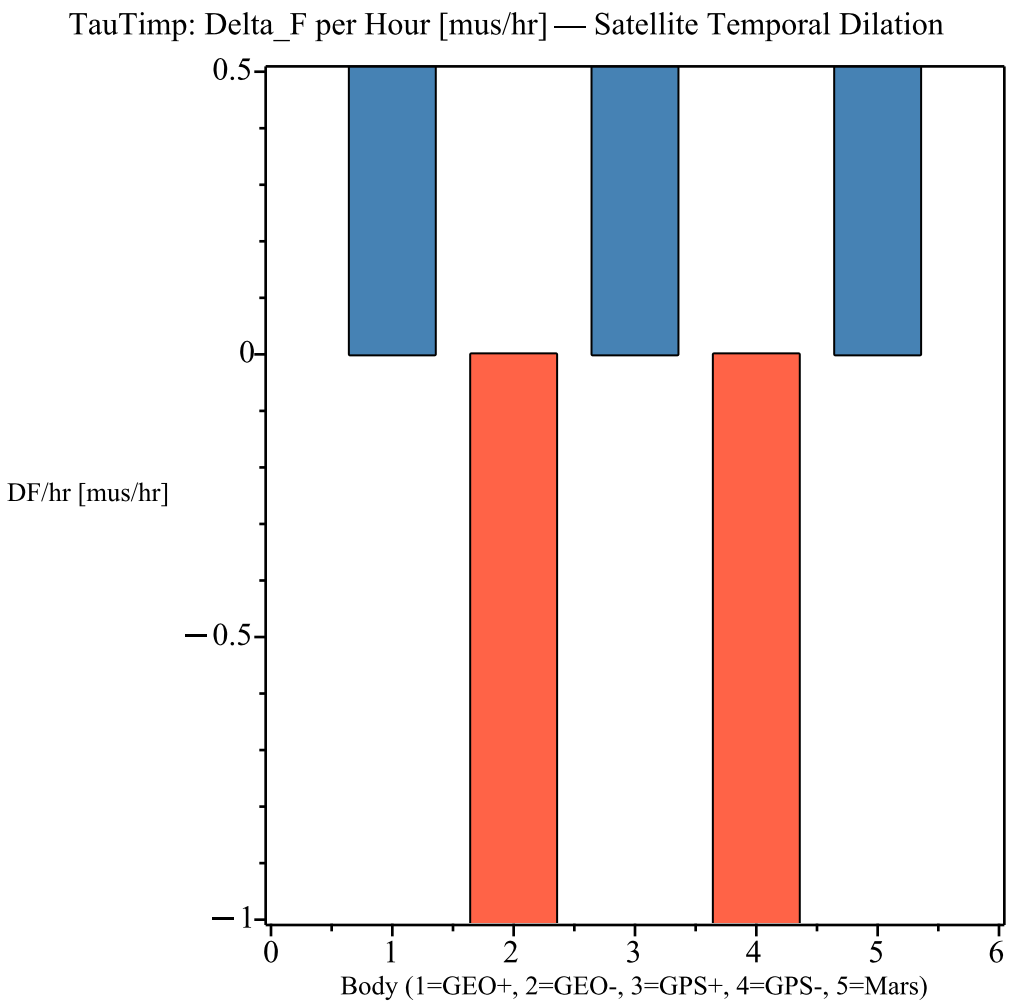
```
GPS+   : DF/hr = +1.4628 mus/hr, DF/day = +35.1073 mus/day
GPS-   : DF/hr = -1.4679 mus/hr, DF/day = -35.2300 mus/day
Mars   : DF/hr = +241.0798 mus/hr, DF/day = +5785.9145 mus/day

=== Natural Units (hbar = c = 1) ===
A_nat   = 4.015403e-19 GeV^2
eps_nat = 5.131371e+06 m = 5.131371e+21 fm

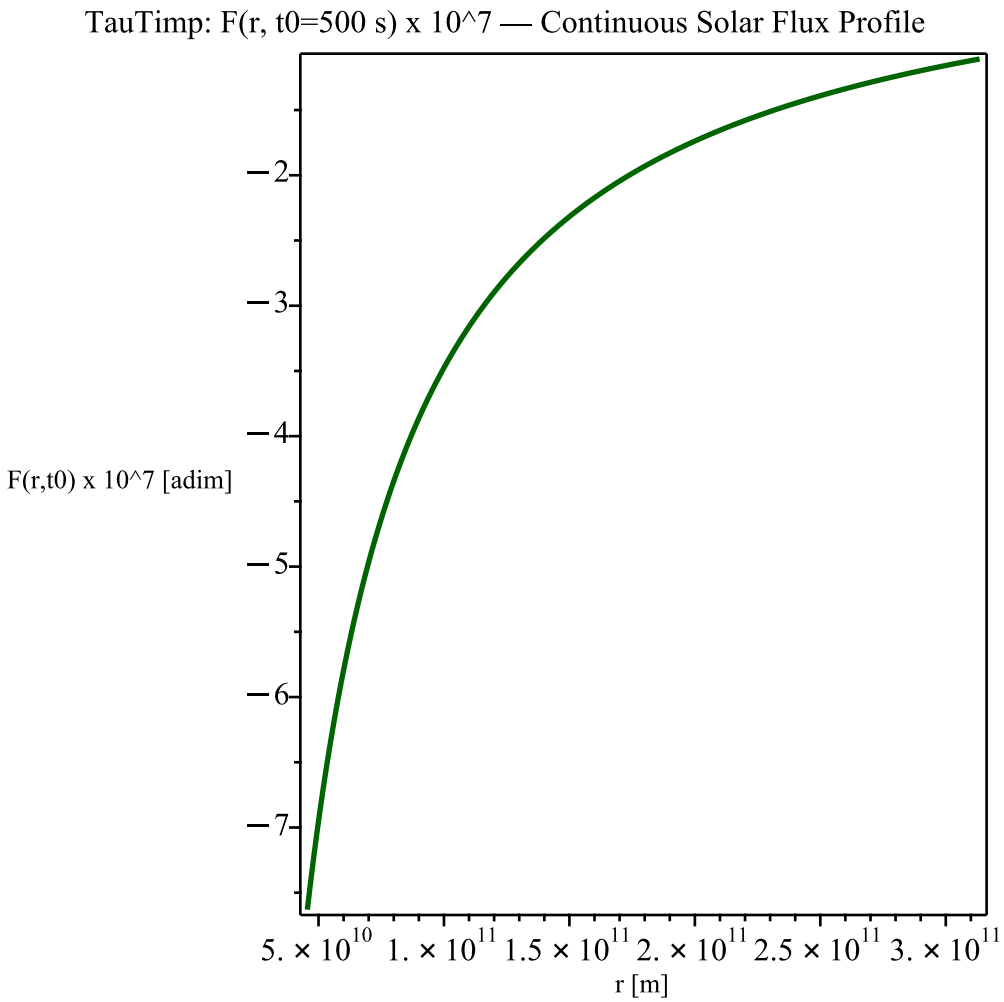
=== Generating Plots ===
```




```
Plot 1 (Solar F profile) — ready
```

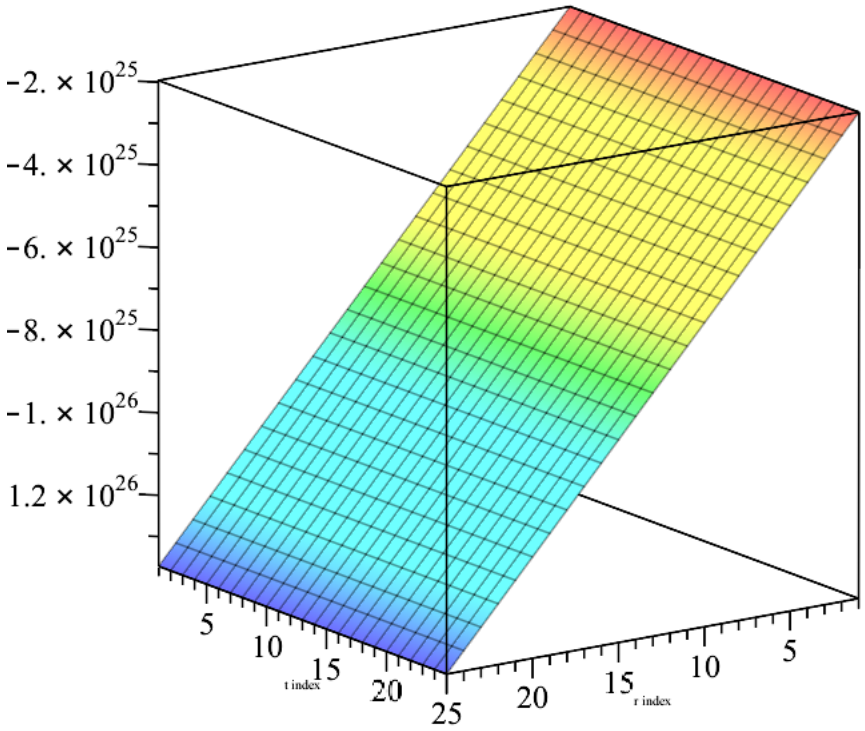


Plot 2 (Satellite bar chart) — ready

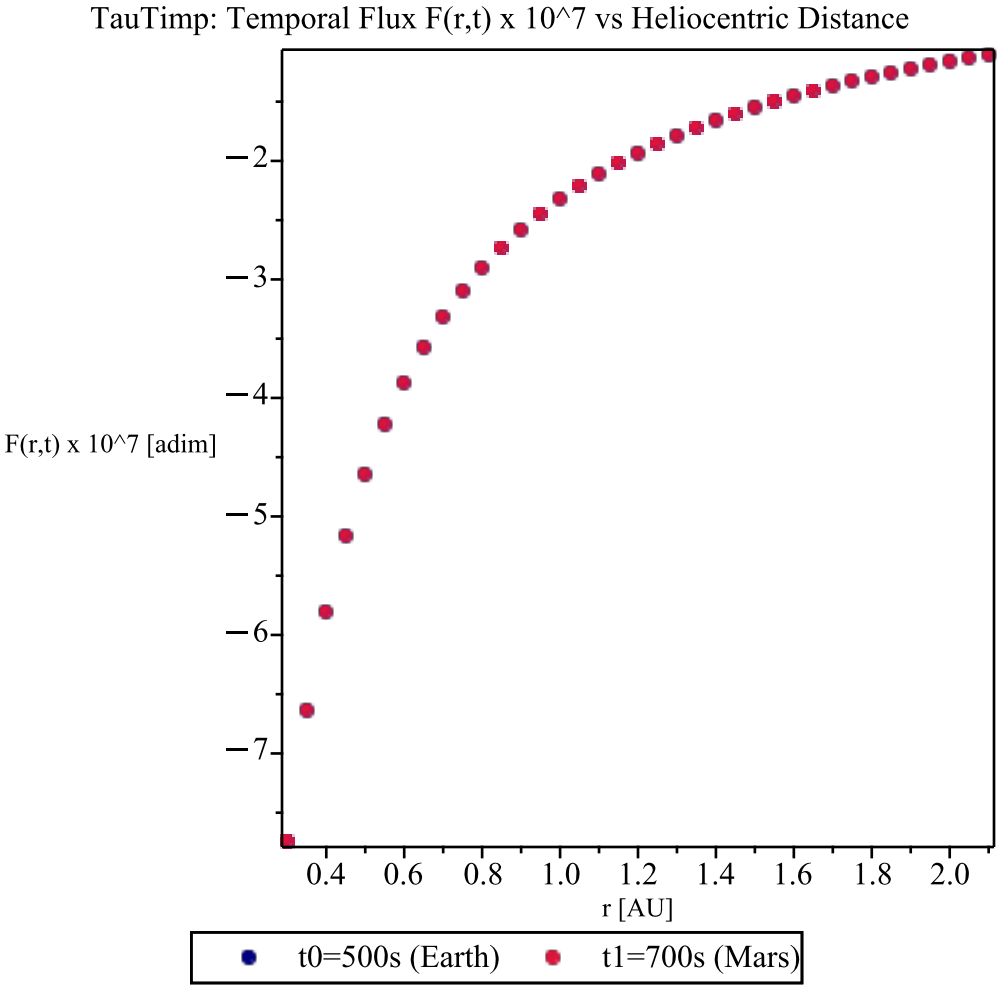


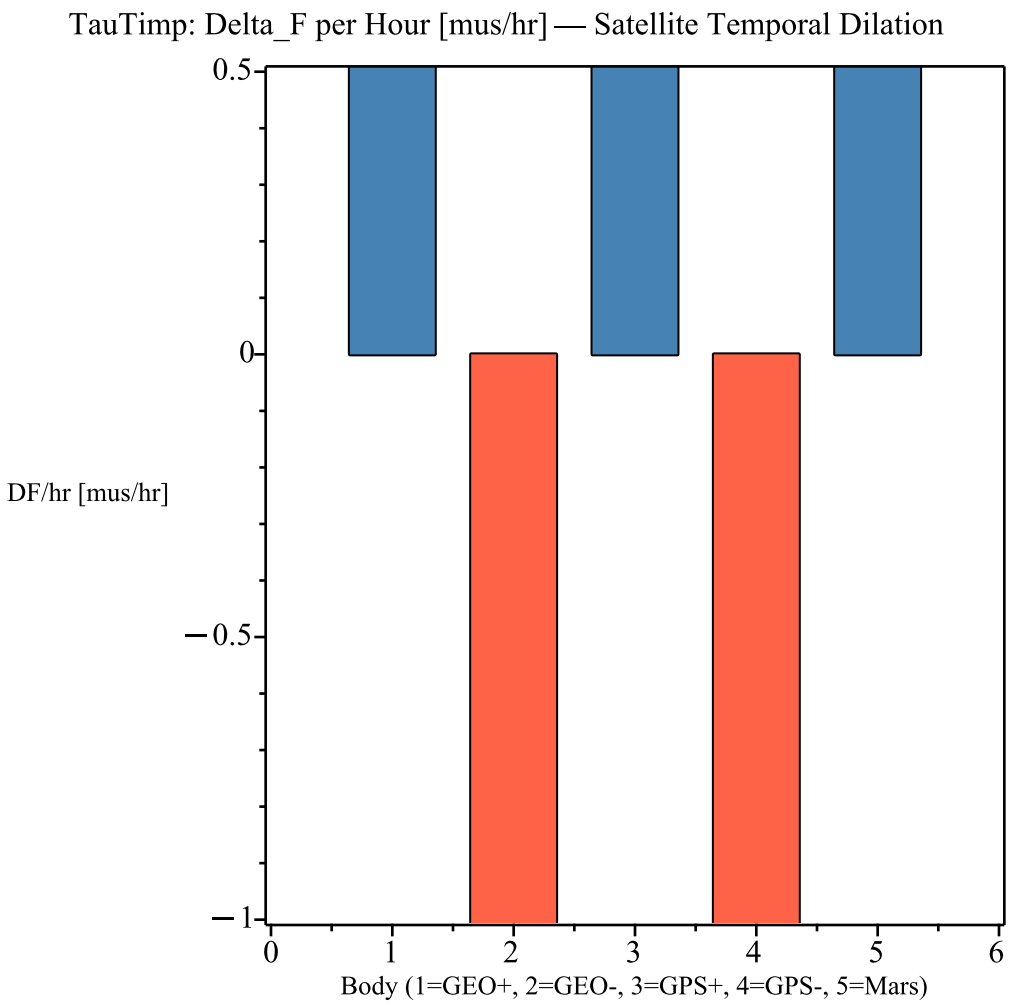
Plot 3 (Continuous F profile) — ready

TauTimp: P(r,t) x 10^9 [3D] — 0.3 to 2.1 AU, t: 0 to 800 s

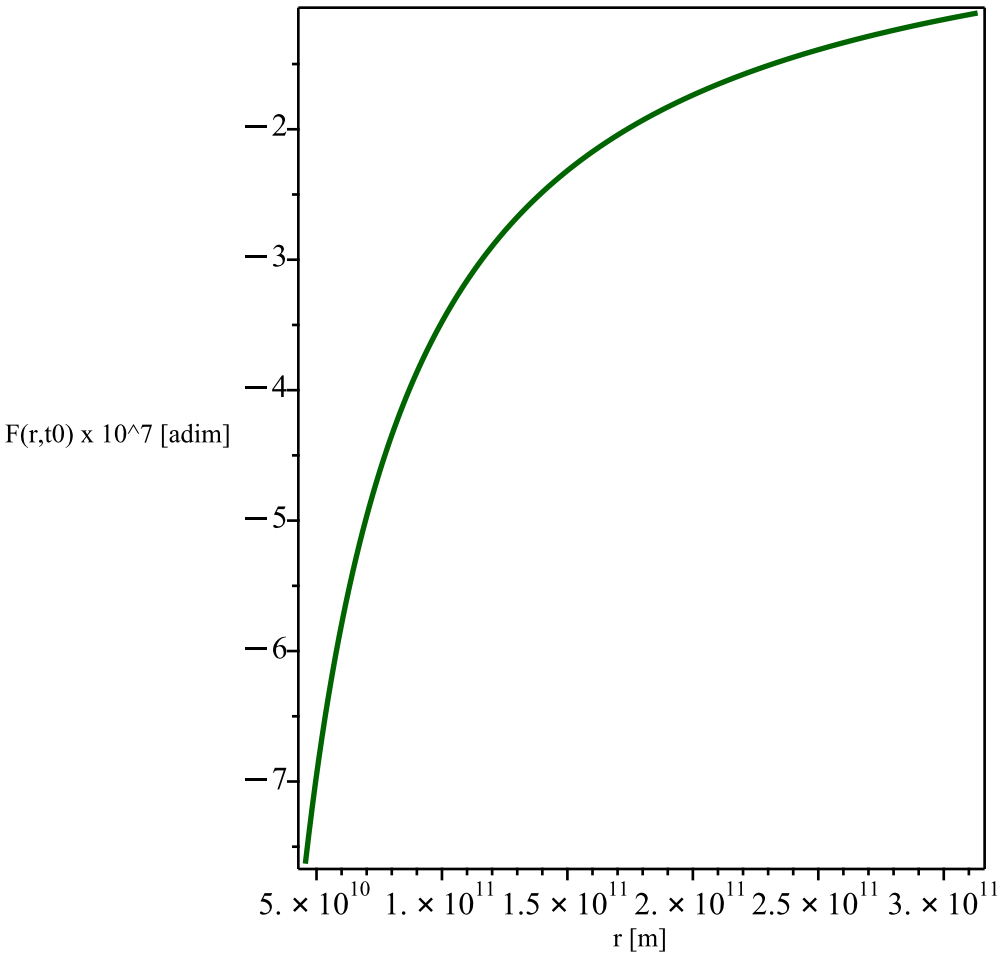


Plot 4 (3D P surface) — ready

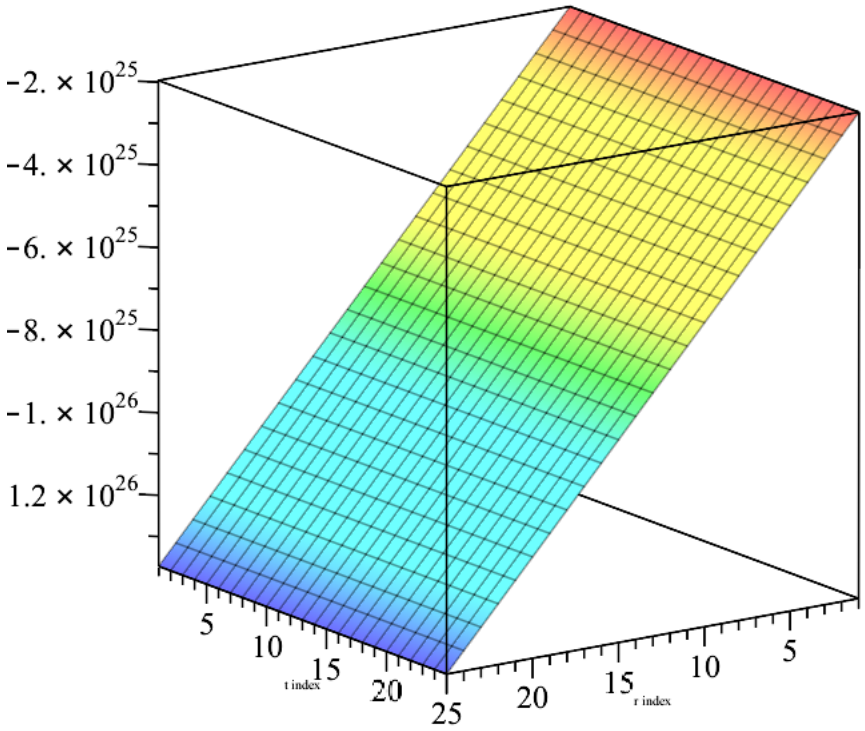

TauTimp: Temporal Flux F(r,t) x 10^7 vs Heliocentric Distance
F(r,t) x 10^7 [adim]
−2
−3
−4
−5
−6
−7
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
r [AU]
t0=500s (Earth)
t1=700s (Mars)


TauTimp: Delta_F per Hour [mus/hr] — Satellite Temporal Dilation
DF/hr [mus/hr]
0.5
0
−0.5
−1
0
1
2
3
4
5
6
Body (1=GEO+, 2=GEO-, 3=GPS+, 4=GPS-, 5=Mars)

TauTimp: F(r, t0=500 s) x 10^7 — Continuous Solar Flux Profile



TauTimp: P(r,t) x 10^9 [3D] — 0.3 to 2.1 AU, t: 0 to 800 s




```
=== All computations complete ===
TauTimp Final Results Summary:
  A      = 1.031200e-17 m^-2
  eps    = 1.538385e+15 m^2/s
  F_Earth= -2.332670e-07
  F_Mars = -1.663004e-07
  GEO+ dilation = +2.3605 mus/hr
  GEO- dilation = -2.3738 mus/hr
  GPS+ dilation = +1.4628 mus/hr
  GPS- dilation = -1.4679 mus/hr
  Mars  dilation = +241.0798 mus/hr
```